Debt-Aware Bonding Curves: Rising Floor Prices and Non-Liquidatable Lending

Ömer Demirel* Floors Finance

Michael Lewkowitz† House Markets

Tiago Santana‡ Floors Finance

Abstract

Decentralized lending protocols rely on liquidation mechanisms tied to volatile oracle-derived prices, creating cascading systemic risk during market downturns. Bonding curve token issuance provides deterministic, oracle-free pricing but has not been integrated with lending to eliminate liquidation. We introduce *debt-aware discrete bonding curves*—piecewise-linear bonding curves augmented with a distinguished floor segment whose price is monotonically non-decreasing. A reserve invariant couples the curve's virtual collateral supply to outstanding debt, enabling a non-liquidatable credit facility in which borrowing capacity is anchored to the endogenous floor price rather than a market oracle. We prove that the floor price is monotonically non-decreasing under all protocol operations and that every loan remains solvent without liquidation. A recursive buy-lock-borrow-buy loop enables non-liquidatable leveraged positions at any time; token launches are the most compelling application, as looping efficiency is highest when the floor-to-spot gap is smallest. We support the mechanism with invariant-preserving curve reconfiguration and demonstrate its properties through stateful invariant-based fuzz testing of a concrete implementation. Comparative analysis shows that this design eliminates oracle dependency, liquidation cascades, and health-factor monitoring from the lending design space.

1 Introduction

1.1 The Liquidation Problem in DeFi

Decentralized lending protocols—most prominently Aave [Aave, 2020], Compound [Leshner and Hayes, 2019], and MakerDAO [MakerDAO, 2017]—enable permissionless borrowing against onchain collateral. These protocols share a common architectural choice: collateral is valued at a market price obtained from an external oracle, and a liquidation mechanism forcibly closes positions whose collateralization ratio falls below a safety threshold.

This design creates several well-documented systemic risks. During periods of rapid price decline, liquidations generate cascading sell pressure that depresses prices further, triggering

*omer@floors.finance

†michael@house.markets

‡tiago@floors.finance

additional liquidations in a self-reinforcing loop [Perez et al., 2021, Tian and Zhu, 2025]. The March 2020 "Black Thursday" event liquidated over $8 million in MakerDAO positions, with some collateral sold for near-zero bids [MakerDAO, 2017]. More recently, research from the Bank for International Settlements has documented how DeFi leverage amplifies volatility transmission across protocols [Heimbach and Huang, 2024]. Liquidation events also attract MEV extraction, with searchers competing to front-run liquidation transactions for profit [Daian et al., 2020].

A natural question arises: can we design a token issuance mechanism in which the collateral's price floor is *contractually guaranteed* and lending against it *provably never requires liquidation*?

1.2 Floor-Price Mechanisms: An Emerging Design Pattern

Several protocols have explored floor-price guarantees as an alternative to liquidation-based lending. Nirvana [Nirvana Finance, 2022] introduced a virtual AMM on Solana with a ratcheting floor price, enabling users to borrow its stablecoin NIRV against ANA tokens valued at the floor. Baseline [Baseline Markets, 2024] manages concentrated liquidity positions on Uniswap V3 across three ranges (Floor, Anchor, Discovery), establishing a Base Liquidity Value (BLV) as a price floor. Olympus [Olympus DAO, 2021] backs its OHM token with a diversified treasury, employing Range Bound Stability (RBS) as an automated monetary policy to defend a backing price. Each approach represents a distinct point in the design space. Virtual AMM-based floors (Nirvana) depend on oracle accuracy and were exploited via flash loan manipulation [Nirvana Finance, 2022]. Range-based strategies (Baseline) depend on the correct operation of external AMMs. Treasury-backed floors (Olympus) are policy-dependent and subject to governance risk.

The present work builds on the discrete bonding curve (DBC) [Demirel, 2023a] and the dynamic discrete bonding curve (DDBC) [Demirel, 2023b], which established stepped bonding curves as gas-efficient, protocol-owned issuance mechanisms [Demirel, 2024b,a]. Unlike secondary-market AMMs (e.g., Uniswap), which trade pre-existing token inventories, a bonding curve acts as a *primary-market* issuer: tokens are minted on purchase and burned on sale, with collateral flowing into and out of a protocol-controlled reserve. This mint/burn architecture is what makes the floor price enforceable—the protocol controls the entire token supply. We term this property *token-owned liquidity*: the reserve backing each token is embedded in its issuance mechanism rather than held in an external treasury or rented from liquidity providers. We extend these foundations with a *debt-aware* reserve invariant and a *floor segment* whose price is provably monotonic, enabling non-liquidatable lending as a direct consequence of the curve's mathematical properties.

1.3 Contributions

This paper makes the following contributions:

1. We formalize *debt-aware discrete bonding curves* (DABC): a class of piecewise-linear bonding curves with a distinguished floor segment whose price is monotonically non-decreasing (Section 3).

2. We state and prove a *reserve invariant* that ensures solvency under all protocol operations, including buying, selling, lending, and curve reconfiguration (Section 3.3).

3. We design a *non-liquidatable credit facility* in which borrowing capacity is anchored to the floor price rather than a volatile market oracle, and prove that no loan can become under-collateralized (Section 4).

4. We introduce a *step-absorption algorithm* for floor elevation that greedily absorbs premium curve steps into the floor segment (Section 3.4).

5. We present *invariant-preserving curve reconfiguration* as a general primitive, enabling arbitrary reshaping of the bonding curve subject only to the reserve invariant (Section 3.5).

6. We verify the mechanism through invariant-based stateful fuzz testing and compare it against five existing protocols (Section 5).

7. We identify *non-liquidatable leveraged positions* as an emergent application: the recursive leverage loop enables participants to amplify exposure without liquidation risk at any point in the curve's lifecycle, with token launches as the most compelling use case, enabling debt-based position design as a structural primitive for token economies (Section 4.5).

2 Related Work

2.1 Bonding Curves and AMMs

Bonding curves define a deterministic relationship between token supply and price, enabling automated market making without external liquidity providers. Bancor [Hertzog et al., 2017] introduced continuous bonding curves with a connector-weight model, where price is a power function of supply. Uniswap V2 [Adams et al., 2020] established the constant-product invariant ($x \cdot y = k$) as the dominant secondary-market AMM design. Uniswap V3 [Adams et al., 2021] introduced concentrated liquidity, allowing liquidity providers to allocate capital within specific price ranges—a technique leveraged by Baseline for floor enforcement. Cartea et al. [Cartea et al., 2024] study strategic bonding curves in AMMs, generalizing constant-function markets to decentralized liquidity pools with configurable impact and quote functions.

The discrete bonding curve (DBC) [Demirel, 2023a] proposed a piecewise-constant price function where price changes at specific supply intervals, yielding zero slippage within each step and eliminating the need for external liquidity providers. The dynamic DBC [Demirel, 2023b] extended this with KPI-based segment reconfiguration, and protocol-owned AMMs [Demirel, 2024b] formalized the concept of protocol-controlled primary issuance. Kirste et al. [Kirste et al., 2025] independently formalized supply-sovereign AMMs with undergirding bonding curves, analyzing value accumulation properties analogous to share-based companies.

Our work extends the DBC family by introducing a floor segment with a formal monotonicity guarantee and a reserve invariant that accounts for outstanding debt—properties absent from prior bonding curve designs.

Table 1: Taxonomy of floor-price mechanisms.

Protocol	Floor mechanism	Guarantee	Lending
Nirvana (ANA)	Virtual AMM + reserves	Algorithmic	NIRV stablecoin
Baseline (YES)	Uniswap V3 ranges	Range-dependent	Non-liquidatable
Olympus (OHM)	Treasury + RBS	Policy-dependent	None
This work	Discrete curve invariant	Formally proven	Non-liquidatable

2.2 Floor-Price Protocols

Table 1 summarizes the floor-price design space.

Nirvana. Nirvana [Nirvana Finance, 2022] implemented a virtual AMM on Solana in which ANA tokens are minted on purchase and burned on sale, with the protocol's reserve backing a ratcheting floor price. Users could borrow the NIRV stablecoin against ANA at the floor value. In July 2022, a flash loan exploit manipulated the virtual AMM's pricing oracle, draining $3.5M from the protocol's reserves. The exploit demonstrated the vulnerability of floor mechanisms that depend on oracle-derived price inputs.

Baseline. Baseline [Baseline Markets, 2024] deploys protocol-owned liquidity across three Uniswap V3 concentrated liquidity positions: a Floor position (dense liquidity at the lowest range), an Anchor position (moderate liquidity bridging to current price), and a Discovery position (wide range for upward price movement). The Base Liquidity Value (BLV) functions as a floor guaranteed by the Floor position's reserves. The mechanism depends on Uniswap V3's correct operation and is subject to the complexity of concentrated liquidity management.

Olympus. Olympus [Olympus DAO, 2021] backs OHM with a diversified treasury and uses Range Bound Stability (RBS) as automated monetary policy. The "liquid backing price" serves as a dynamic floor, with the protocol buying OHM when the price falls below this threshold. BlockScience modeled the system using cadCAD simulations [Olympus DAO, 2021]. The floor guarantee is policy-dependent: it holds only as long as treasury assets retain their value and governance maintains the RBS parameters.

2.3 DeFi Lending and Liquidation

Aave [Aave, 2020] and Compound [Leshner and Hayes, 2019] are overcollateralized lending protocols where collateral is marked to market via external oracles. When the health factor (collateral value divided by debt) falls below a threshold, liquidators can repay a portion of the debt in exchange for discounted collateral. Perez et al. [Perez et al., 2021] conducted an empirical study showing that liquidations exhibit strong procyclical effects. The Bank of Canada [Tian and Zhu, 2025] analyzed how liquidation mechanism design—fixed-spread versus auction-based—affects price impact, finding that auction-based liquidations reduce cascading effects when participation costs are low.

MakerDAO [MakerDAO, 2017] uses Collateralized Debt Positions (CDPs) with keeper-based liquidation auctions. Curve's crvUSD [Egorov, 2023] introduced LLAMMA, a soft

liquidation mechanism that continuously converts collateral into the borrowed asset as prices decline, avoiding discrete liquidation events.

A separate line of work removes oracle dependency but retains liquidation or forced conversion. Timeswap [Timeswap Labs, 2022] uses a three-variable AMM ($X \cdot Y \cdot Z = K$) with fixed-maturity loans; non-repayment at maturity constitutes default and collateral forfeiture. Ajna [Ajna Labs, 2023] implements peer-to-pool lending with discrete price buckets and no external oracle, but liquidation is triggered when the market price falls below a borrower's bucket price. Both designs demonstrate that removing oracles alone does not eliminate liquidation.

Transaction fee mechanism design [Roughgarden, 2021] provides a broader theoretical framework for analyzing incentive-compatible fee structures in blockchain protocols.

All existing lending protocols require some form of liquidation, forced conversion, or maturity-based default when collateral values decline. Our mechanism eliminates this requirement entirely by anchoring LTV to a monotonically non-decreasing endogenous price, removing the need for external oracles, liquidation infrastructure, and fixed loan maturities.

3 Model

We now formalize the discrete bonding curve mechanism, the reserve function that underpins solvency, the floor-elevation algorithm, and the reconfiguration principle that makes the curve a *configurable liquidity primitive*. Throughout, we denote by $\mathbb{Z}_{\geq 0}$ the non-negative integers.

Primary-market semantics. The bonding curve operates as a *primary-market* issuer. Buying mints new tokens and deposits collateral into the reserve; selling burns tokens and withdraws collateral. Unlike secondary-market AMMs (e.g., Uniswap constant-product pools), which trade pre-existing token inventories, the protocol controls the entire token supply. This is what makes the floor price enforceable: because every token was minted through the curve, every token can be redeemed through it.

3.1 Discrete Bonding Curve Segments

Definition 3.1 (Segment)**.** A *segment* is a tuple

$$\sigma = (p_0,\ \Delta p,\ q,\ n),$$

where $p_0 \in \mathbb{Z}_{>0}$ is the *initial price*, $\Delta p \in \mathbb{Z}_{\geq 0}$ the *price increment per step*, $q \in \mathbb{Z}_{>0}$ the *supply per step*, and $n \in \mathbb{Z}_{>0}$ the *number of steps*. The *price at step* i ($0 \leq i < n$) is

$$p(i) \ = \ p_0 + i\,\Delta p, \tag{1}$$

and the *segment capacity* is $C(\sigma) = n \cdot q$. A segment is *flat* when $\Delta p = 0$; it is *sloped* otherwise. If $n = 1$, the segment must be flat ($\Delta p = 0$).

Definition 3.2 (Discrete Bonding Curve)**.** A *discrete bonding curve* is an ordered sequence of segments

$$\Gamma = (\sigma_0,\, \sigma_1,\, \ldots,\, \sigma_{k-1}), \qquad k \leq K_{\max},$$

satisfying the *price-progression constraint*: for every adjacent pair $\sigma_i = (p_0^{(i)}, \Delta p^{(i)}, q^{(i)}, n^{(i)})$ and $\sigma_{i+1} = (p_0^{(i+1)}, \ldots)$,

$$p_0^{(i)} + (n^{(i)} - 1)\,\Delta p^{(i)} \;\le\; p_0^{(i+1)}. \tag{2}$$

The *total curve capacity* is $C(\Gamma) = \sum_{j=0}^{k-1} C(\sigma_j)$.

3.2 Floor Segment and Reserve Function

Definition 3.3 (Floor Curve)**.** A curve $\Gamma = (\sigma_0, \sigma_1, \ldots, \sigma_{k-1})$ is a *floor curve* if the distinguished first segment σ_0 satisfies $n_0 = 1$ and $\Delta p_0 = 0$. The constant $P_f \coloneqq p_0^{(0)}$ is the *floor price* and $S_0 \coloneqq q_0$ is the *floor supply*.

The floor segment establishes a flat price band at the base of the curve. Because $n_0 = 1$ and $\Delta p_0 = 0$, every token within the floor segment is redeemable at exactly P_f.

Definition 3.4 (Reserve Function)**.** Let $\Gamma = (\sigma_0, \ldots, \sigma_{k-1})$ be a discrete bonding curve and let S be the current token supply with $0 \le S \le C(\Gamma)$. The *reserve function* $R(\Gamma, S)$ gives the total collateral required to back supply S. It is computed segment by segment.

For a single segment $\sigma = (p_0, \Delta p, q, n)$ processing a supply portion s $(0 \le s \le C(\sigma))$, let $m = \lfloor s/q \rfloor$ (full steps) and $r = s \bmod q$ (partial-step remainder).

(i) **Flat segment** ($\Delta p = 0$):

$$R_\sigma(s) \;=\; s \cdot p_0. \tag{3}$$

(ii) **Sloped segment** ($\Delta p > 0$). The full-step cost uses the arithmetic-series identity:

$$R_\sigma^{\text{full}}(m) \;=\; \frac{q \cdot m\,(2\,p_0 + (m-1)\,\Delta p)}{2}, \tag{4}$$

and the partial-step cost at step m is

$$R_\sigma^{\text{part}}(r) \;=\; r \cdot (p_0 + m\,\Delta p). \tag{5}$$

The total segment reserve is $R_\sigma(s) = R_\sigma^{\text{full}}(m) + R_\sigma^{\text{part}}(r)$.

The global reserve is computed by iterating over segments in order, allocating $\min(s_j, C(\sigma_j))$ supply to each segment σ_j until all supply is assigned:

$$R(\Gamma, S) \;=\; \sum_{j=0}^{k-1} R_{\sigma_j}\big(\min(s_j,\, C(\sigma_j))\big), \tag{6}$$

where $s_0 = S$ and $s_{j+1} = s_j - \min(s_j, C(\sigma_j))$; that is, s_j is the supply not yet assigned when processing segment σ_j.

Unit conventions. All formulas are stated in abstract units in which *price* × *quantity* = *collateral* directly. Prices (p_0, Δp, P_f), token amounts (S, q, ℓ), and reserve/debt quantities (R, V, d) are commensurable without an explicit scaling factor.

Implementation mapping. The Solidity implementation maps these abstract quantities to fixed-point integer arithmetic: prices are stored in WAD ($\times 10^{18}$), token amounts in token-wei (18 decimals), and collateral in collateral-wei. Every price–quantity product is divided by 10^{18} using a `mulDivUp` helper (ceiling, protocol-favorable) for collateral computations and floor division for token computations, ensuring the contract never under-collateralizes at wei-level precision.

3.3 The Solvency Invariant

The protocol maintains a *virtual collateral supply* V, an onchain counter that tracks the total collateral backing the issued tokens.

Invariant 3.1 (Solvency)**.** At every state transition,

$$V \;=\; R(\Gamma,\, S), \tag{7}$$

where S is the current token supply and Γ the active curve.

Here V is the *virtual* collateral supply, an onchain counter. During lending, the contract's *actual* reserve balance may be lower: $\text{bal} = V - \sum_i d_i$, where d_i are outstanding loan debts. The locked issuance tokens backing those loans have a floor-price value of at least $\sum_i d_i$, so the full reserve can be restored upon repayment (see Section 4).

Invariant 3.1 is enforced by the reconfiguration check (Section 3.5): any proposed curve Γ' is accepted if and only if $R(\Gamma', S) = V$.

Lemma 3.1 (Reserve Lower Bound)**.** *For any floor curve Γ with floor price P_f and any $x \geq 0$, $R(\Gamma, x) \geq x \cdot P_f$.*

Proof. By the monotone ordering constraint (Definition 3.2), every segment σ_j has $p_0^{(j)} \geq P_f$. Each unit of supply is therefore priced at least P_f. □

Proposition 3.1 (Redemption Liquidity)**.** *Let $D = \sum_i d_i$ denote the total outstanding debt and $A = V - D$ the actual reserve balance held by the contract. Let $S_{\text{free}} = S - \sum_i \ell_i$ denote the unlocked (freely tradable) supply. Locked tokens cannot be sold or transferred; only unlocked supply may be burned for collateral. The protocol enforces this by holding locked tokens in custody: they are transferred to the credit facility contract at loan creation and returned only upon repayment. No approval, delegation, or wrapping mechanism can bypass this custody. Then the actual reserves are sufficient to cover any sell or redemption of unlocked tokens:*

$$A \;\geq\; R(\Gamma,\; S) - R(\Gamma,\; S - S_{\text{free}}).$$

Proof. By Theorem 4.2, each active loan satisfies $d_i \leq \gamma \cdot P_f \cdot \ell_i < P_f \cdot \ell_i$ (since $\gamma < 1$). Summing over all loans gives $D < \sum_i \ell_i \cdot P_f$. By Lemma 3.1, the reserve value of the locked supply satisfies $R(\Gamma, S - S_{\text{free}}) \geq \sum_i \ell_i \cdot P_f > D$. Since $V = R(\Gamma, S)$, we have $A = V - D = R(\Gamma, S) - D$. The maximum payout for selling all unlocked supply is $R(\Gamma, S) - R(\Gamma, S - S_{\text{free}})$. Combining: $A = R(\Gamma, S) - D \geq R(\Gamma, S) - R(\Gamma, S - S_{\text{free}})$. □

Proposition 3.2 (Redemption Preserves Floor)**.** *If x tokens are redeemed at the floor price P_f, the post-redemption floor ratio is unchanged:*

$$\frac{A_f - x \cdot P_f}{S_0 - x} \;=\; P_f,$$

where $A_f = P_f \cdot S_0$ denotes the floor-tier collateral.

Proof. By the floor-segment definition, $A_f = P_f \cdot S_0$. After redeeming x tokens:

$$\frac{A'_f}{S'_0} = \frac{P_f \cdot S_0 - x \cdot P_f}{S_0 - x} = \frac{P_f\,(S_0 - x)}{S_0 - x} = P_f.$$

The equality holds in exact arithmetic; with integer-arithmetic rounding (floor division), rounding cannot decrease the enforced floor price parameter P_f, since P_f is a configured segment price rather than a value computed from A_f/S_0 onchain. □

3.4 Floor Elevation: Step-Absorption Algorithm

When external collateral is injected (e.g., from protocol fees or surplus reallocation), the floor price can be raised by *absorbing* premium steps into the floor segment. Algorithm 1 formalizes this procedure.

The algorithm operates in two modes. In *normal mode* ($S_0 < S_{\text{actual}}$), absorbing a step increases both the floor price and the floor supply. In *capped mode* ($S_0 = S_{\text{actual}}$), the floor supply is frozen at S_{actual} and only the price increases; the denominator S_{eff} remains fixed, ensuring that collateral is never spent backing unminted tokens. The *capped* flag is one-way: once set, it cannot be cleared within the same invocation.

Post-absorption reserve reconciliation. In abstract units the loop arithmetic is exact. The Solidity implementation, operating in fixed-point integers, may accumulate rounding errors across iterations; it therefore recomputes the exact collateral requirement $c_{\text{actual}} = R(\Gamma', S) - V$ via the reserve function (Definition 3.4) *after* the loop, and transfers only c_{actual}. This two-phase design ensures Invariant 3.1 holds regardless of intermediate rounding in the integer implementation.

Harmonic step sizing. In normal mode, each absorbed step adds its capacity q_j to S_0. The collateral cost of raising the floor by one price unit is proportional to S_0, so unchecked growth of the floor supply makes future elevations progressively more expensive. If all steps have equal capacity q, absorbing m steps yields $S_0 \propto mq$ and the cumulative cost scales as $O(m^2)$. A natural mitigation is to set step capacities that decrease with price level—for instance, $q_j \propto 1/j$ (harmonic sizing). Under harmonic capacities, S_0 after absorbing m steps grows as the harmonic sum $\sum_{j=1}^{m} 1/j = O(\log m)$, and the cumulative elevation cost scales as $O(m \log m)$ rather than $O(m^2)$. To see this, note that each absorption costs $\Delta P_f \cdot S_0$ collateral (the step gap times the current floor supply). With uniform q, $S_0 = \Theta(m)$ after m absorptions, so cumulative cost is $\Theta(\sum_{j=1}^{m} j) = \Theta(m^2)$. With $q_j \propto 1/j$, $S_0 = \Theta(\log m)$, yielding cumulative cost $\Theta(m \log m)$. This ensures that long-term floor appreciation remains feasible even after many steps have been absorbed.

Proposition 3.3 (Floor Elevation Correctness)**.** *Let Γ' be the curve produced by Algorithm 1 with collateral injection c. Then:*

(a) $P'_f \geq P_f$ *(floor price is non-decreasing);*

(b) $S'_0 \geq S_0$ *(floor supply is non-decreasing);*

(c) $R(\Gamma', S) = V + c_{\text{actual}}$*, where* $c_{\text{actual}} \leq c$ *is the collateral actually consumed (accounting for rounding).*

Proof sketch. (a) Every iteration either increases P_f by the full gap to the next step or by a positive residual *raisable*; neither branch decreases P_f. (b) S_0 is only modified by addition (line 16); the one-way cap (lines 17–18) ensures $S_0 \leq S_{\text{actual}}$ at all times. (c) The post-loop reconciliation computes $c_{\text{actual}} = R(\Gamma', S) - V$ exactly via the reserve function (Definition 3.4) and transfers only this amount, ensuring Invariant 3.1 holds after the update. □

3.5 Invariant-Preserving Curve Reconfiguration

Prior work introduced discrete (stepped) bonding curves as a gas-efficient alternative to continuous price functions Demirel [2023a], and subsequently demonstrated that segments can be dynamically reconfigured in response to external performance indicators Demirel [2023b]. The solvency invariant (Equation 7) suggests a strict generalization of this approach: the curve may be reconfigured *arbitrarily*—not only in response to specific KPIs—provided the reserve invariant is preserved.

Definition 3.5 (Valid Reconfiguration)**.** Given current state (V, S, Γ) and a configured *maximum spot-loss parameter* $\delta_{\max} \geq 0$, a new curve Γ' is a *valid reconfiguration* if and only if

(i) **Reserve invariance:**

$$R(\Gamma',\ S)\ =\ V. \tag{8}$$

If additional collateral δ is supplied alongside the reconfiguration, the check becomes $R(\Gamma', S) = V + \delta$.

(ii) **Bounded spot-price loss:** for every supply level $s \leq S$ (i.e., within the circulating supply range),

$$P(\Gamma',\ s)\ \geq\ P(\Gamma,\ s) - \delta_{\max}, \tag{9}$$

where $P(\Gamma, s)$ denotes the step price at supply level s.

Setting $\delta_{\max} = 0$ makes the reconfiguration *spot-price non-dilutive*: no minted token may have its price reduced. A positive $\delta_{\max}$ permits gradual reshaping within a known bound.

The bounded spot-loss constraint prevents a single reconfiguration from redistributing premium-tier value to the floor. Without it, the reserve invariant alone admits curve flattenings that preserve $R(\Gamma', S) = V$ while collapsing the spot price—a vector by which a reconfiguration could inflate floor-locked borrowing capacity at the expense of premium holders. Token holders should be aware that reconfiguration may reduce spot prices by up to $\delta_{\max}$ per invocation; this parameter is part of the curve's public configuration and should be set conservatively (e.g., a small fraction of the current floor price) or fixed at zero for maximum holder protection. Note that $\delta_{\max}$ bounds a *single* invocation; repeated reconfigurations can accumulate dilution beyond this bound. Pairing $\delta_{\max}$ with epoch-based invocation limits or a cumulative budget per time window prevents unbounded dilution through rapid successive calls.

Floor elevation (Algorithm 1) is a special case of reconfiguration that satisfies both constraints by construction: it only increases prices within the circulating range, and the algorithm computes Γ' such that $\delta = R(\Gamma', S) - V$, with the caller supplying exactly δ.

Proposition 3.4 (Reconfiguration Safety)**.** *Any reconfiguration satisfying Definition 3.5 preserves:*

(i) Solvency*: Invariant 3.1 holds for* Γ'*;*

(ii) Floor monotonicity*: the floor is non-decreasing (*$P'_f \geq P_f$*), enforced as an onchain precondition;*

(iii) Full redemption coverage*: every token holder can redeem at or above the floor price;*

(iv) Bounded dilution*: no minted token's step price decreases by more than* $\delta_{\max}$*.*

Proof. (i) holds by construction of the reserve check. (ii) is enforced as an onchain precondition (the transaction reverts if $P'_f < P_f$). (iii) follows from (i): the virtual collateral V exactly covers the reserve requirement $R(\Gamma', S)$, which in turn covers all redemptions down to the floor tier. (iv) follows from the spot-loss check (Equation 9), which is verified for every supply level within the circulating range. □

This reconfiguration principle elevates the bonding curve from a static price–supply schedule to a *configurable liquidity primitive*. Whereas the original DBC design Demirel [2023a] fixes segments at deployment, and the DDBC extension Demirel [2023b] permits KPI-triggered adjustments, the present work removes the trigger constraint entirely: reshaping of the premium tiers is valid—floor elevation, liquidity reallocation toward the floor, adaptive curve reshaping—so long as Definition 3.5 is satisfied. We refer to authorized reconfigurations as *Liquidity Reallocation Events (LREs)*.

3.6 Why Discrete? Continuous vs. Discrete Bonding Curves

The choice of a *discrete* (stepped) bonding curve over a continuous one (e.g., polynomial or exponential) is deliberate and motivated by the following considerations.

1. **Deterministic pricing.** Within each step, the price is constant. Buyers and sellers know the exact price before submitting a transaction; there is no slippage within a step.

2. **Natural floor segment.** A flat step ($\Delta p = 0$, $n = 1$) maps directly to a guaranteed floor price. Continuous curves have no canonical flat region and require auxiliary mechanisms to enforce a price floor.

3. **Step-absorption tractability.** Raising the floor reduces to iterating over a small number of discrete steps (Algorithm 1), each requiring only integer arithmetic. A continuous analogue would require solving integrals onchain—prohibitively expensive in gas.

4. **Onchain computability.** Reserve computations use only addition, multiplication, and integer division (with explicit rounding). No transcendental functions (ln, exp, fractional exponents) are needed, eliminating fixed-point approximation errors.

Table 2: Discrete vs. continuous bonding curves.

Property	**Discrete**	**Continuous**
Intra-step slippage	None	Proportional to order size
Floor segment	Native (flat step)	Requires auxiliary contract
Onchain math	Integer arithmetic	Transcendental functions
Floor elevation	Step iteration	Integral solving
Rounding control	Per-step boundaries	Distributed error
Fuzz/formal verification	Finite state space	Infinite state space
Granularity trade-off	Arbitrarily small steps	Inherently smooth

5. **Composability with lending.** The discrete structure confines rounding discrepancies to step boundaries, simplifying the invariant checks required when the credit facility (Section 4) withdraws or deposits collateral.

6. **Formal verifiability.** Discrete state spaces are amenable to exhaustive fuzz testing and bounded model checking. The reference implementation has been verified against segment-level invariants under randomized inputs.

7. **Approximation of continuity.** The loss of granularity is negligible in practice: by choosing sufficiently small step sizes (q and Δp), the discrete curve can approximate any continuous price function to arbitrary precision while retaining all the above advantages.

Table 2 summarizes the trade-offs.

4 Non-Liquidatable Lending

We now introduce the credit facility that allows token holders to borrow the reserve asset against their holdings. The key design insight is that borrowing capacity is evaluated at the *floor price*—not the market price—and the floor is monotonically non-decreasing. Consequently, no protocol-triggered liquidation mechanism is required. Concretely, no protocol operation forcibly seizes or sells a borrower's locked tokens. Borrowers repay voluntarily to unlock their position; if they choose not to repay, the tokens remain locked indefinitely. The consequence of non-repayment is permanent lock, not forced sale.

4.1 Credit Facility Design

Definition 4.1 (Loan)**.** A *loan* is a tuple

$$L \;=\; (\mathit{borrower},\ \ell,\ d,\ \mathit{active}),$$

where *borrower* is the owner address, $\ell \in \mathbb{Z}_{>0}$ is the number of locked issuance tokens, $d \in \mathbb{Z}_{>0}$ is the outstanding debt denominated in the reserve asset, and $\mathit{active} \in \{\mathit{true}, \mathit{false}\}$ records whether the loan is open.

Definition 4.2 (Borrowing Power)**.** Given ℓ locked tokens, a loan-to-value ratio $\gamma_{\text{bps}} \in [1, 10{,}000)$ (equivalently $\gamma = \gamma_{\text{bps}}/10{,}000 \in (0, 1)$), and the current floor price P_f, the *maximum borrowing power* is

$$\text{maxBorrow}(\ell) \;=\; \gamma \cdot P_f \cdot \ell. \tag{10}$$

Key design choice. Borrowing power is computed against the *floor price* P_f, not the spot (market) price. Since P_f is monotonically non-decreasing (Theorem 4.1), borrowing power can only stay constant or increase over time—never decrease due to market volatility. The LTV ratio γ_{bps} may be adjusted by governance; lowering it affects only future borrows, not existing loan safety (see Theorem 4.2). The one-time origination fee $\phi > 0$ is charged at loan creation; no streaming interest accrues. This is not merely a simplification—it is a *prerequisite* for non-liquidatable safety. If interest accrued continuously at rate r, the outstanding debt would grow as $d(t) = d_0\, e^{r(t-t_0)}$, and Theorem 4.2 would require $\dot{P}_f/P_f \geq r$ at all times. Since floor elevation depends on fee revenue that is neither guaranteed nor predictable, no protocol can ensure this bound under all market conditions. By fixing d at origination, the debt-to-collateral ratio can only *improve* as P_f rises, making non-liquidatable safety a permanent invariant rather than a conditional guarantee.

Collateral flow. When a loan is originated, the credit facility calls `withdrawCollateralTo` on the bonding curve to obtain reserve tokens without reducing the virtual collateral supply V. On repayment, `depositCollateralFrom` returns reserve tokens without increasing V. This preserves the solvency invariant (Invariant 3.1) across lending operations: the virtual supply V remains unchanged, while the *physical* reserve balance temporarily decreases by the amount lent out. The locked issuance tokens serve as a claim that guarantees the reserve can be restored upon repayment.

Non-rivalrous borrowing capacity. Unlike pooled lending protocols (Aave, Compound) where all borrowers draw from a shared liquidity pool—and one borrower's utilization reduces the capacity available to others—this credit facility provides *non-rivalrous* borrowing capacity. Each minter's locked tokens represent collateral they individually contributed to the reserve, and their borrowing power is derived solely from their own position via Equation (10). One borrower's loan does not diminish another's credit limit. This follows from the per-loan accounting structure (Definition 4.1): each loan's (ℓ, d) pair is independent, and the aggregate reserve impact is the sum of individual withdrawals. Total debt is bounded by the reserve value of locked supply, which preserves solvency and redemption coverage of unlocked tokens (Proposition 3.1). Borrow execution, however, is subject to the contract's current collateral balance and may revert if insufficient liquidity is available at that moment.

4.2 Floor Price Monotonicity

Theorem 4.1 (Floor Price Monotonicity)**.** *Under the protocol operations* buy*,* sell*,* raiseFloor*, and* reconfigure *(subject to the onchain precondition $P_f' \geq P_f$), the floor price P_f is monotonically non-decreasing:*

$$P_f^{(t+1)} \;\geq\; P_f^{(t)} \qquad \textit{for all time steps } t.$$

Proof. We analyze each operation individually.

(i) **Buy.** A purchase adds collateral to V and mints tokens (increasing S) along premium segments. The floor segment σ_0 is not modified; hence P_f is unchanged.

(ii) **Sell.** A sale burns tokens and returns collateral proportional to the reserve difference $R(\Gamma, S) - R(\Gamma, S - x)$. If selling reduces the total supply S below the floor supply S_0, the protocol invokes AdjustFloorToSupply, which sets $S_0' = S$ while keeping P_f fixed:

$$\sigma_0' \;=\; (P_f,\; 0,\; S,\; 1).$$

This adjustment ensures that any subsequent minting resumes on premium segments rather than the floor, preventing floor-supply dilution. The floor backing ratio is preserved as a side effect (Proposition 3.2): $\lfloor A_f'/S_0' \rfloor \geq P_f$ due to rounding. Hence P_f does not decrease.

(iii) **raiseFloor.** By Proposition 3.3(a), the step-absorption algorithm only increases P_f.

(iv) **Reconfigure.** By Proposition 3.4, any valid reconfiguration preserves solvency. The implementation enforces $P_f' \geq P_f$ as an onchain precondition (the transaction reverts otherwise). □

4.3 Non-Liquidatable Safety

Theorem 4.2 (Non-Liquidatable Safety)**.** *An active loan $L = (\mathit{borrower}, \ell, d, \mathit{true})$ satisfies $d < P_f \cdot \ell$ for all $t \geq t_{\text{creation}}$. That is, the floor-price value of the locked tokens always exceeds the outstanding debt, regardless of any subsequent changes to γ_{bps}.*

$$d \;\leq\; \text{maxBorrow}(\ell) \;=\; \gamma \cdot P_f \cdot \ell \;<\; P_f \cdot \ell \qquad \textit{for all } t \geq t_{\text{creation}}. \tag{11}$$

Consequently, the protocol requires no liquidation mechanism.

Proof. We establish the invariant by induction on protocol events.

Base case (loan creation). At origination, the required issuance tokens are computed as

$$\ell \;\geq\; \frac{d}{\gamma \cdot P_f},$$

which satisfies $d \leq \text{maxBorrow}(\ell) = \gamma \cdot P_f \cdot \ell$ by construction.

Inductive step—partial repayment. When the borrower repays an amount $r < d$, the protocol unlocks tokens proportionally:

$$\ell' \;=\; \ell - \left\lfloor \frac{r \cdot \ell}{d} \right\rfloor, \qquad d' \;=\; d - r.$$

We verify:

$$\frac{d'}{\ell'} \;=\; \frac{d - r}{\ell - \lfloor r\ell/d \rfloor} \;\leq\; \frac{d}{\ell},$$

where the inequality follows because $\lfloor r\ell/d \rfloor \leq r\ell/d$, so the denominator shrinks by at most $r\ell/d$ while the numerator shrinks by exactly r. Since $d/\ell < P_f$ held before repayment (by $\gamma < 1$), it continues to hold after, preserving Equation (11).

Table 3: Comparison of lending designs.

Property	**Proposed mechanism**	**Aave / Compound**	**MakerDAO**
Collateral valuation	Floor price P_f	Spot oracle	Spot oracle
Price monotonicity	Guaranteed	Not guaranteed	Not guaranteed
Liquidation mechanism	None required	Health-factor auction	Keeper auction
Interest accrual	None (one-time fee)	Variable / stable rate	Stability fee
Loan term	Open-ended	Open-ended	Open-ended
Oracle dependency	None (endogenous P_f)	External price feeds	External price feeds

Inductive step—floor elevation. By Theorem 4.1, P_f can only increase. Since $d < P_f \cdot \ell$ at origination (because $\gamma < 1$), and P_f only grows, the inequality $d < P_f' \cdot \ell$ is strengthened at every floor raise. The debt d remains fixed between borrower actions; therefore the loan becomes *better* collateralized over time (*passive de-risking*).

LTV parameter changes. Even if γ_{bps} is lowered after origination, the bound $d < P_f \cdot \ell$ is γ-independent and continues to hold. A reduction in γ affects only future borrows, not existing loan safety.

No other events modify loan state. Buy, sell, and reconfiguration operations do not alter the locked tokens ℓ or the debt d of any loan. Combined with floor monotonicity, this ensures Equation (11) is a global invariant. □

Comparison with existing protocols. Table 3 contrasts the proposed credit facility with established DeFi lending protocols.

4.4 Recursive Leverage and Bounds

The credit facility enables a *recursive leverage* strategy: a participant can buy tokens, lock them, borrow against the floor, and use the proceeds to buy more tokens. This buy→lock→borrow→buy loop can be iterated to amplify exposure.

Let γ denote the LTV ratio and ϕ the effective fee rate per iteration, encompassing the origination fee and any trading fees incurred during the buy step. The *reinvestment fraction* per loop is

$$\eta \;=\; \gamma\,(1-\phi). \tag{12}$$

Since each loop reinvests a fraction η of the previous round's position, the total leverage follows a geometric series.

Proposition 4.1 (Leverage Bound)**.** *The net leverage achievable through recursive looping is*

$$L_{\text{net}} \;=\; \sum_{i=0}^{\infty} \eta^i \;=\; \frac{1}{1-\eta} \;=\; \frac{1}{1-\gamma(1-\phi)}, \tag{13}$$

provided $\eta < 1$, which holds whenever $\gamma < 1$ or $\phi > 0$.

Proof. At loop i, the participant's incremental position is proportional to η^i times the initial deposit. The sum converges to a standard geometric series. The constraint $\eta < 1$ is ensured by the protocol: $\gamma_{\text{bps}} < 10{,}000$ (i.e., $\gamma < 1$), giving $\eta = \gamma(1-\phi) \le \gamma < 1$. □

Table 4: Net leverage $L_{\text{net}} = 1/(1-\gamma(1-\phi))$ for selected parameter combinations.

γ (LTV)	ϕ (fee)	$\eta = \gamma(1-\phi)$	L_{net}
0.90	0.03	0.873	7.87×
0.90	0.04	0.864	7.35×
0.85	0.03	0.8245	5.70×
0.80	0.03	0.776	4.46×
0.95	0.03	0.9215	12.74×

Proposition 4.2 (Leverage Decay under Discrete Steps)**.** *Let $P^{(i)}_{\text{spot}}$ denote the effective purchase price at loop iteration i. Because each successive buy traverses higher-priced premium steps, the per-iteration reinvestment fraction decays:*

$$\eta_i \;=\; \gamma\,(1-\phi)\,\frac{P_f}{P^{(i)}_{\text{spot}}}, \qquad \eta_i \;\le\; \eta_{i-1} \;\le\; \eta_0 = \gamma(1-\phi). \tag{14}$$

The realized leverage is therefore strictly bounded above by the geometric series:

$$L_{\text{realized}} \;=\; 1 + \sum_{i=0}^{N} \prod_{k=0}^{i} \eta_k \;<\; \frac{1}{1-\eta_0} \;=\; L_{\text{net}}.$$

This monotonic decay provides an algorithmic containment of leveraged supply expansion: a participant cannot achieve unbounded leverage because the DBC's natural slippage compresses their effective LTV on each sequential purchase.

Proof. At origination, the borrowing capacity is $\gamma \cdot P_f \cdot \ell$. The proceeds are reinvested at $P^{(i)}_{\text{spot}} \ge P_f$, yielding $\ell_{i+1} \propto P_f / P^{(i)}_{\text{spot}}$ tokens per unit of capital. Since the DBC is monotonically non-decreasing in price along the supply axis (Definition 3.2), $P^{(i+1)}_{\text{spot}} \ge P^{(i)}_{\text{spot}}$, giving $\eta_{i+1} \le \eta_i$. The product $\prod_{k=0}^{i} \eta_k \le \eta_0^i$ yields $L_{\text{realized}} = 1 + \sum_{i=0}^{N} \prod_{k=0}^{i} \eta_k < 1 + \sum_{i=0}^{\infty} \eta_0^i = 1/(1-\eta_0)$. □

Remark. Proposition 4.2 is a *strength* of the DBC architecture: leveraged supply expansion is algorithmically self-limiting without requiring external circuit breakers.

Example. With $\gamma = 0.90$ and $\phi = 0.03$:

$$\eta = 0.90 \times 0.97 = 0.873, \qquad L_{\text{net}} = \frac{1}{1-0.873} \approx 7.87\times .$$

Table 4 shows leverage sensitivity to parameter choices.

Safety at each iteration. The coverage check (Invariant 3.1) is verified at every loop iteration. If the remaining collateral is insufficient to maintain the solvency invariant after a proposed borrow, the loop terminates early. This bounds the actual leverage to at most L_{net} and prevents system insolvency even under adversarial looping.

4.5 Application: Leveraged Token Launches

The recursive leverage mechanism enables non-liquidatable leveraged positions at any point in the curve's lifecycle. Token launches are the most compelling application: the floor-to-spot gap is smallest early in the curve, maximizing looping efficiency, and no external oracle exists for newly issued tokens.

The problem. In oracle-based DeFi, a participant wishing to amplify exposure to a newly launched token follows the same buy-deposit-borrow-buy loop through Aave or Compound. However, the collateral is valued at the *spot oracle price*, which for a newly launched token is maximally volatile. A downward price movement triggers liquidation—the participant loses both the position and a liquidation penalty [Perez et al., 2021].

Floor-anchored leverage. In our mechanism, borrowing capacity is evaluated against P_f, not the spot price. Because P_f is monotonically non-decreasing (Theorem 4.1) and every loan remains safe (Theorem 4.2), *no loan in the loop can ever become under-collateralized*, regardless of subsequent market-price movements—even if the market price falls to the floor itself. The total position converges to $C_0 \cdot L_{\text{net}}/P_f$ tokens, where C_0 is initial capital and $L_{\text{net}} = 1/(1-\gamma(1-\phi))$. At every iteration, the collateral buffer—the excess of floor-valued collateral over outstanding debt—*grows* with each subsequent floor elevation.

Anti-front-running for presale leverage. During presale periods, a time-decaying fee multiplier is applied to origination and trading fees. The multiplier follows a quartic decay from an initial premium (e.g., 5×) to the base rate (1×) over the presale duration:

$$\mu(t) \;=\; 1 + (\mu_0 - 1)\left(\frac{t_{\text{end}} - t}{t_{\text{end}} - t_{\text{start}}}\right)^4,$$

where μ_0 is the initial multiplier. The quartic exponent on remaining time causes fees to decay aggressively in the early presale window, shedding over 90% of the premium by the midpoint, then converging slowly to 1× over the remainder—discouraging front-running by ensuring the highest fees are encountered immediately upon presale opening.

Fee-funded bootstrapping. The one-time origination fee serves a dual role in this context: beyond pricing credit risk, it functions as an *algorithmic bootstrapping mechanism*. Each leverage loop iteration generates origination fees that are directed to the floor reserve, funding subsequent floor elevation. This replaces the inflationary token emissions commonly used to bootstrap DeFi protocols [Heimbach and Huang, 2024]: rather than diluting existing holders to attract liquidity, the mechanism channels leverage demand into floor appreciation that benefits all token holders. The result is a non-dilutive bootstrapping loop in which early leveraged participation directly strengthens the protocol's price-floor guarantee.

5 Analysis and Evaluation

We evaluate the debt-aware bonding curve mechanism along five axes: comparative positioning against related floor-price designs (§5.1), capital efficiency properties (§5.2), invariant-

based verification of a concrete implementation (§5.3), failure-mode analysis (§5.4), and an agent-based simulation study of dynamic fee multipliers (§5.5).

5.1 Comparative Analysis

Table 5 positions the present work against five representative systems that incorporate floor-price or reserve-backed token mechanics.

Table 5: Comparison of floor-price and reserve-backed token designs. • = present, ○ = absent, △ = partial.

Property	**This work**	**Nirvana**	**Baseline**	**Olympus**	**Aave**
Floor enforcement	•	•	•	○	○
Guarantee strength	Reserve inv.	Algorithmic	Range LP	Treasury	N/A
Lending integration	•	△	△	○	•
Oracle dependency	○	•	△	•	•
Floor monotonicity	Proven	Claimed	○	○	N/A
Curve reconfigurability	•	○	○	○	N/A

Floor enforcement. Nirvana [Nirvana Finance, 2022] and Baseline [Baseline Markets, 2024] enforce a floor through algorithmic stabilization and concentrated-range Uniswap V3 positions, respectively. Our mechanism enforces the floor through a discrete curve segment with an onchain reserve invariant $V = R(\Gamma, S)$ (Invariant 3.1). Proposition 3.1 guarantees that actual reserves ($V - D$, where D is aggregate outstanding debt) cover all redemptions of unlocked supply. This makes the guarantee constructive rather than emergent.

Guarantee strength. Olympus [Olympus DAO, 2021] relies on treasury backing whose value fluctuates with external asset prices. Our reserve invariant ensures that the actual collateral held by the contract, net of outstanding debt, always covers the floor segment's obligation, providing a deterministic, per-token guarantee.

Lending integration. Aave [Aave, 2020] is the canonical oracle-based lending protocol: collateral is marked to market and positions are liquidated when health factors breach a threshold. No other floor-price protocol integrates a credit facility. Our design anchors loan-to-value to the floor price P_f rather than the spot price, eliminating the need for liquidation entirely.

Oracle dependency. Our mechanism requires no external price oracle. The floor price is an endogenous quantity derived from the reserve invariant and the current segment configuration. Every other lending-capable system in Table 5 depends on an oracle for collateral valuation, creating an external attack surface.

Floor monotonicity. We prove (Theorem 4.1) that P_f is non-decreasing under all protocol operations. Baseline does not provide a comparable monotonicity guarantee. Nirvana claims a rising floor but the guarantee is algorithmic rather than invariant-based.

Curve reconfigurability. Our reconfiguration primitive (Definition 3.5) allows reshaping the premium segments of Γ while preserving all invariants, including floor monotonicity and reserve solvency. No comparable mechanism exists in related work.

5.2 Capital Efficiency Analysis

LTV at floor versus market price. In oracle-based protocols, the loan-to-value ratio is computed against the current spot price P_{spot}, which can decline precipitously. In our design the LTV is computed against P_f:

$$\text{maxBorrow} \;=\; \gamma \cdot P_f \cdot S_{\text{locked}}$$

where γ is the protocol-configured LTV ratio and S_{locked} is the quantity of locked issuance tokens. Because $P_f \le P_{\text{spot}}$ by construction, the effective utilization at origination is conservative. However, as the floor rises the borrower's capacity *increases without additional collateral*, enabling rebalancing—extracting additional collateral from the same locked position.

Effective capital utilization improves as the floor rises. Let $P_f(t_0)$ and $P_f(t_1)$ denote the floor price at loan origination and a later time $t_1 > t_0$ respectively. The borrowing capacity grows by the factor $P_f(t_1)/P_f(t_0) \ge 1$, while the original debt remains fixed. At origination, $D_0 \le \text{maxBorrow} = \gamma \cdot P_f(t_0) \cdot S_{\text{locked}}$, and since no interest accrues, $D_{\text{remaining}} \le D_0$. The effective LTV at time t_1 is therefore:

$$\text{LTV}_{\text{eff}}(t_1) \;=\; \frac{D_{\text{remaining}}}{P_f(t_1) \cdot S_{\text{locked}}} \;\le\; \frac{D_0}{P_f(t_1) \cdot S_{\text{locked}}} \;\le\; \gamma \cdot \frac{P_f(t_0)}{P_f(t_1)} \;\le\; \gamma$$

Hence the loan becomes *more* collateralized over time without any action from the borrower, a property we term *passive de-risking*.

Antifragility of the floor. Redemptions that reduce the total supply below the floor supply trigger AdjustFloorToSupply, which reduces S_0 to match. Since the cost to raise the floor by one price tick is proportional to S_0, a smaller floor supply makes future floor raises cheaper. Concretely, if the treasury allocates a fraction α of each trade's fee (at rate f) to floor reserves, the annual floor lift is approximately

$$\Delta P_f^{\text{annual}} \;\approx\; \frac{V_{\text{daily}} \cdot f \cdot \alpha \cdot 365}{S_0}$$

where V_{daily} is average daily trading volume (all quantities in base units; the result is in collateral-wei per token-wei, i.e., a raw price increment, not WAD-scaled). Bear-market redemptions shrink S_0, accelerating the floor's rise for remaining holders—a form of antifragility.

5.3 Invariant-Based Verification

We verify the mechanism's critical properties through stateful invariant-based fuzz testing of a concrete Solidity implementation named *Floors*, comprising approximately 20,000 lines of Solidity across 30+ contracts, licensed under LGPL-3.0.

Testing framework. The verification suite uses the Foundry stateful fuzzing engine. Handler contracts wrap each protocol module (bonding curve, credit facility, presale, access control) and expose fuzzer-callable entry points that execute bounded, randomized sequences of protocol operations. Ghost variables in each handler track cumulative collateral flows, issuance minting and burning, and loan lifecycle state, enabling the test harness to assert invariants after every operation sequence.

Key invariants tested. The following invariants are checked after every fuzz-generated operation sequence:

1. **Reserve solvency.** The virtual collateral supply V_c matches the reserve computed from the current segment configuration and total issuance supply, within a tolerance of 100 wei: $|V_c - R(\Gamma, \text{totalSupply})| \leq 100$. The protocol maintains exact equality ($V_c = R(\Gamma, S)$) by recomputing the reserve function atomically; the tolerance is a conservative detection margin for the fuzz harness, not an acknowledgment of drift.

2. **Floor monotonicity.** The floor price P_f never falls below its value at deployment. A dedicated ghost variable tracks the high-water mark and asserts $P_f(t) \geq P_f(0)$ for all t.

3. **Segment validity.** The curve always has at least two segments. The floor segment has exactly one step with zero price increment. Every segment has positive supply per step, positive step count, and positive initial price.

4. **Collateral backing.** The actual collateral token balance of the curve contract, plus cumulative withdrawals, equals the virtual collateral supply plus cumulative deposits: $\text{bal} + \Sigma_{\text{withdrawn}} = V_c + \Sigma_{\text{deposited}}$. This accounting identity ensures no collateral is created or destroyed.

5. **Loan safety.** Every active loan satisfies $D_i \leq \gamma \cdot P_f \cdot S_i^{\text{locked}}$, where D_i and S_i^{locked} are the i-th loan's remaining debt and locked issuance tokens, respectively. The sum of locked tokens across all active loans equals the credit facility's issuance token balance.

Workflow-level testing. Beyond per-module invariant tests, a full-protocol workflow handler orchestrates multi-step scenarios—buy-and-borrow, full loan lifecycles (borrow $\rightarrow$ rebalance $\rightarrow$ repay), price appreciation through floor raises, leveraged loop positions, and concurrent multi-user operation sequences. System-wide invariants (full backing guarantee, floor monotonicity) are checked after every workflow invocation.

Test infrastructure. The handler base contract provides actor management (up to 2^{16} deterministic addresses), bounded time and block warping, and realistic amount bounding. All handlers operate under `fail_on_revert = true`, meaning the fuzzer treats unexpected reverts as failures rather than silently discarding them.

5.4 Failure Mode Analysis

We distinguish between properties that *can* be violated under certain conditions and those that *cannot* be violated under a correct implementation.

What can break.

- **Smart contract bugs.** Implementation errors could violate any invariant. The fuzz-testing suite mitigates but does not eliminate this risk. Formal verification of the core curve mathematics and reserve accounting is left as future work.
- **Operator misconfiguration.** Premium segments may be reshaped within the bound $\delta_{\max}$, and floor raises may be delayed. Onchain validation constraints prevent any reconfiguration from lowering the floor or violating the reserve invariant.
- **Collateral token failures.** The mechanism assumes a well-behaved ERC-20 collateral token. Fee-on-transfer tokens would cause the actual collateral balance to diverge from the virtual supply, violating the reserve invariant. Rebasing tokens would similarly break the accounting identity. The implementation explicitly does not support these token types.

What cannot break (under correct implementation).

- **Floor monotonicity.** The floor price P_f is non-decreasing by construction. `raiseFloor` can only increase P_f. Redemptions at the floor leave P_f unchanged (Theorem 4.1). `reconfigureSegments` enforces $P_f^{\text{new}} \geq P_f^{\text{old}}$ as a precondition.
- **Reserve solvency.** The virtual collateral supply is updated atomically with every buy and sell. Credit facility withdrawals and deposits do not alter the virtual supply, preserving the curve invariant. Segment reconfiguration checks $R(\Gamma', S) = V$ (or $V + \delta$ if collateral is injected) before committing (Definition 3.5).
- **Loan safety.** Because the LTV is anchored to P_f and P_f is non-decreasing, no active loan can become under-collateralized through market movements. The protocol contains no liquidation mechanism; repayment is the only path to loan closure.

Trust assumptions. The mechanism assumes: (i) the smart contract implementation is correct, (ii) the collateral token conforms to the ERC-20 standard (an implementation constraint, not a fundamental limitation), and (iii) the underlying blockchain provides liveness and finality. Onchain preconditions ensure the floor can never be lowered; floor raises are the only operation that requires periodic invocation, and automating them through time-locked triggers or programmatic schedules is straightforward.

5.5 Dynamic Fee Multiplier: Simulation Study

The discussion in Section 6 identifies dynamic fee multipliers as a natural extension of the mechanism. We now present a concrete instantiation and evaluate it through agent-based simulation.

5.5.1 Fee Model

We decompose the effective fee into a static base fee f_{base} and a state-dependent multiplier m:

$$f_{\text{eff}} = f_{\text{base}} \cdot \frac{m}{10{,}000} \tag{15}$$

where m is expressed in basis points (BPS; 10,000 = 1×). The multiplier is the product of two independent components:

Premium multiplier $h(\pi)$. Let $\pi = P_{\text{spot}}/P_{\text{floor}}$ denote the premium ratio. We define asymmetric buy-side and sell-side multipliers using Michaelis–Menten kinetics:

$$h_{\text{buy}}(\pi) = 1 + \alpha_{\text{buy}} \cdot \frac{\pi - 1}{\pi - 1 + c_{\text{buy}}} \tag{16}$$

$$h_{\text{sell}}(\pi) = 1 + \alpha_{\text{sell}} \cdot \frac{c_{\text{sell}}}{\max(0,\ \pi - 1) + c_{\text{sell}}} \tag{17}$$

where α controls the saturation ceiling and c the half-saturation constant. The buy-side multiplier h_{buy} saturates at $1 + \alpha_{\text{buy}}$ for large premiums, incentivizing buying near the floor. The sell-side multiplier h_{sell} is maximal at the floor ($\pi = 1$, yielding $1 + \alpha_{\text{sell}}$) and decays toward 1 at high premiums, penalizing sells near the floor to protect its integrity.

Size multiplier $g(s)$. Let $s = q_{\text{last}}/S$ denote the ratio of the most recent trade size to the total supply, lagged by one trade. The size multiplier applies a quadratic penalty:

$$g(s) = \min\left(1 + \beta \cdot \frac{s^2}{s_{\text{ref}}^2},\ g_{\max}\right) \tag{18}$$

where β scales the penalty, s_{ref} is the reference size, and $g_{\max}$ caps the multiplier. The quadratic scaling penalizes large trades super-linearly while remaining negligible for small trades. The one-step lag ensures the multiplier is deterministic at execution time.

Combined multiplier. The final multipliers for buy and sell are:

$$m_{\text{buy}} = h_{\text{buy}} \cdot g, \qquad m_{\text{sell}} = h_{\text{sell}} \cdot g \tag{19}$$

capped at 15× (150,000 BPS) to bound worst-case fees.

Rationale. This two-input model was selected through an ablation study over a five-component multiplicative fee framework (premium, size, volatility, spread, and momentum). The two-input variant $h(\pi) \cdot g(s)$ captures 95.8% of the full model's protocol revenue while requiring only BPS integer arithmetic (additions, multiplications, divisions—no transcendentals) and a single EVM storage slot ($8 \times$ `uint32` $= 256$ bits) for all parameters.

5.5.2 Simulation Setup

We evaluate seven fee strategies in an agent-based simulation framework with heterogeneous traders (informed, noise, and arbitrage agents) operating on a discrete bonding curve with floor elevation.

- **Static**: Fixed symmetric fees ($f_{\text{buy}} = f_{\text{sell}} = 300$ BPS).
- **Premium-aware**: Fees scale linearly with the premium ratio π.
- **Volatility-responsive**: Fees increase with realized volatility.
- **Spread-aware**: Fees adjust based on bid-ask spread.
- **Asymmetric accumulator**: Low buy fees (150 BPS), high sell fees (500 BPS).
- **Adaptive hybrid**: Combines premium and volatility signals.
- **Floors Dynamic**: The two-input model defined above, with parameters $\alpha_{\text{buy}} = 4.0$, $c_{\text{buy}} = 0.2$, $\alpha_{\text{sell}} = 1.5$, $c_{\text{sell}} = 0.3$, $\beta = 4.0$, $s_{\text{ref}} = 5\%$, $g_{\max} = 3.0$.

Each strategy is evaluated across six market regimes—low volatility, high volatility, bull trend, bear trend, mean-reverting, and liquidity shock—using 20 independent random seeds per regime (common random numbers for pairwise comparisons), yielding 840 total simulations of 10,000 steps each.

5.5.3 Results

Protocol revenue. Table 6 reports the mean protocol revenue (collateral units) across 20 seeds per regime. The Floors Dynamic strategy ranks first in all six regimes.

Table 6: Mean protocol revenue ± standard deviation across 20 seeds. Bold indicates the highest revenue per regime. Floors Dynamic ranks #1 in all six regimes.

Regime	Static	Prem.-aware	Asym. accum.	Adapt. hybrid	Floors Dyn.
Low vol.	116.8 ± 49.9	107.6 ± 45.5	122.0 ± 50.1	112.2 ± 49.4	$\mathbf{137.3 \pm 64.3}$
High vol.	261.3 ± 206.4	275.0 ± 227.6	266.4 ± 205.9	271.9 ± 223.2	$\mathbf{309.1 \pm 235.9}$
Bull trend	608.9 ± 78.4	673.0 ± 45.6	483.9 ± 76.8	571.2 ± 49.1	$\mathbf{711.5 \pm 65.1}$
Bear trend	24.9 ± 13.9	20.2 ± 12.2	30.0 ± 19.0	18.3 ± 11.3	$\mathbf{38.2 \pm 21.4}$
Mean-rev.	102.7 ± 4.2	101.4 ± 1.0	104.3 ± 2.3	102.2 ± 1.2	$\mathbf{107.4 \pm 5.3}$
Liq. shock	224.0 ± 118.4	256.5 ± 146.9	193.4 ± 108.5	240.6 ± 132.7	$\mathbf{263.6 \pm 133.3}$
Grand avg.	223.1	238.9	200.0	219.4	**261.2**

Across all regimes, Floors Dynamic achieves a grand average revenue of 261.2 collateral units, representing a +17.1% uplift over Static (223.1) and a +9.3% uplift over Premium-aware (238.9), the next-best strategy.

Table 7: Statistical significance of Floors Dynamic vs. competitors. Mean uplift in collateral units, 95% bootstrap CI, and Cohen's d. All CIs exclude zero except the one marked with †.

Regime	**vs.**	**Mean uplift**	**95% CI**	**Cohen's d**
Low vol.	Static	+20.4	[5.2, 37.4]	0.54
	Premium-aware	+29.6	[13.7, 47.1]	0.75
High vol.	Static	+47.8	[26.8, 72.7]	0.88
	Premium-aware	+34.0	[5.4, 65.9]	0.48
Bull trend	Static	+102.6	[83.3, 120.0]	2.38
	Premium-aware	+38.5	[17.5, 60.4]	0.77
Bear trend	Static	+13.3	[8.7, 18.5]	1.16
	Premium-aware	+18.0	[12.9, 23.7]	1.43
Mean-rev.	Static	+4.7	[1.5, 7.8]	0.63
	Premium-aware	+6.0	[3.9, 8.5]	1.11
Liq. shock	Static	+39.6	[27.8, 51.3]	1.42
	Premium-aware†	+7.2	[−11.3, 22.9]	0.18

Statistical significance. Table 7 reports pairwise comparisons using common random numbers (CRN) with bootstrapped 95% confidence intervals and Cohen's d effect sizes.

The uplift is statistically significant in 11 of 12 pairwise comparisons. The sole exception is Floors Dynamic vs. Premium-aware under the liquidity shock regime ($d = 0.18$), where the CI includes zero; note, however, that Floors Dynamic still achieves a higher point estimate.

Fee asymmetry. Table 8 illustrates the adaptive fee asymmetry produced by the model. The buy–sell fee split adjusts endogenously to market conditions: bull markets see elevated buy fees (capturing surplus from eager buyers) with reduced sell fees (encouraging holding), while bear markets exhibit the reverse pattern, sharply penalizing sells near the floor.

Table 8: Average effective buy and sell fees (BPS) for the Floors Dynamic strategy across regimes. Fees are asymmetric and regime-dependent.

Regime	**Buy fee**	**Sell fee**	**Dominant side**
Low vol.	450	929	Sell-heavy
High vol.	475	697	Sell-heavy
Bull trend	531	297	Buy-heavy
Bear trend	480	1013	Sell-heavy
Mean-rev.	499	933	Sell-heavy
Liq. shock	427	564	Sell-heavy

The sell-heavy pattern in most regimes reflects the floor-protection mechanism of h_{sell}: when the premium ratio π is close to 1, sell fees spike to deter extraction near the floor. In bull trends, the premium is large ($\pi \gg 1$), so h_{sell} decays toward 1 while h_{buy} saturates upward, reversing the asymmetry.

Arbitrage leakage. The higher revenue of Floors Dynamic comes with increased arbitrage activity: arbitrageurs extract an average of 34.1 collateral units across all regimes, compared to 3.5 for Static. This is a design-aware trade-off: the dynamic fees intentionally widen the effective spread in certain states, creating round-trip arbitrage opportunities. However, the net effect on protocol revenue remains strongly positive, as the fee revenue captured from arbitrageurs exceeds the leakage. In the bear trend regime—where floor protection is most critical—both strategies exhibit zero arbitrage leakage.

Floor price appreciation. Table 9 reports the average number of floor raises and floor price appreciation per regime. Floors Dynamic achieves the highest number of floor raises in the bull trend regime (6.50 vs. 5.55 for Static), consistent with its higher fee revenue funding more frequent floor elevation.

Table 9: Average floor raises (count) and floor price appreciation (ΔP_f) per regime.

	Static		**Floors Dynamic**	
Regime	Raises	ΔP_f	Raises	ΔP_f
Low vol.	0.90	0.205	1.05	0.171
High vol.	2.20	0.208	2.65	0.208
Bull trend	5.55	0.338	6.50	0.316
Bear trend	0.00	0.000	0.00	0.000
Mean-rev.	1.00	0.211	1.00	0.186
Liq. shock	1.65	0.194	2.15	0.205

Head-to-head win rates. Using common random numbers across all 120 seed–regime pairs, Floors Dynamic wins the head-to-head protocol revenue comparison against every competitor with the following rates: 91.7% vs. Static, 84.2% vs. Premium-aware, 95.0% vs. Spread-aware, 88.3% vs. Adaptive hybrid, 83.3% vs. Asymmetric accumulator, and 80.8% vs. Volatility-responsive.

5.5.4 Onchain Implementation

The two-input fee model is implemented as a reusable abstract Solidity contract (`DynamicFeeMultiplierBa`
All arithmetic is performed in basis points using OpenZeppelin's `Math.mulDiv` for precision. The eight `uint32` parameters pack into a single 256-bit storage slot, adding approximately 8,000 gas overhead per trade (two `SLOAD` operations plus BPS arithmetic). The combined multiplier is capped at 15× (150,000 BPS) to bound worst-case effective fees. The contract is standalone: it does not modify base fees, presale fees, origination fees, or any existing contract in the protocol.

Fixed-point encoding. The saturation ceilings α_{buy}, α_{sell} and half-saturation constants c_{buy}, c_{sell} are stored as `uint32` values scaled by 10,000 (BPS), so $\alpha_{\text{buy}} = 4.0$ is stored as 40,000 and $c_{\text{buy}} = 0.2$ as 2,000. The premium ratio $\pi = P_{\text{spot}}/P_f$ is computed inline in

10^{18}-scaled (WAD) arithmetic and converted to BPS before entering the multiplier. The size multiplier parameters β, s_{ref}, and $g_{\max}$ use the same BPS encoding. This mapping from the continuous formulas (Equations 16–18) to integer arithmetic is exact up to rounding at each `mulDiv` step.

5.5.5 Discussion

The simulation results demonstrate that the Michaelis–Menten premium multiplier combined with quadratic size scaling constitutes a Pareto-improving fee policy: protocol revenue increases in every tested regime without degrading floor integrity. The asymmetric fee structure emerges naturally from the model's functional form—h_{sell} peaks at $\pi = 1$ and decays, while h_{buy} starts at 1 and saturates—rather than from hard-coded asymmetry, making it adaptive to regime changes without parameter tuning.

The increased arbitrage leakage is a consequence of wider effective spreads in high-multiplier states. This represents a deliberate revenue–leakage trade-off: the protocol sacrifices some value to arbitrageurs in exchange for substantially higher fee revenue from organic traders. In production, the parameters α, c, β, and s_{ref} can be tuned via governance to adjust this trade-off. Setting $\alpha_{\text{buy}} = \alpha_{\text{sell}} = 0$ and $\beta = 0$ recovers the static fee baseline.

6 Discussion and Limitations

Reconfiguration bounds. Onchain preconditions ensure that no reconfiguration can lower the floor price or violate the reserve invariant. Reshaping premium segments can reduce spot prices for existing holders within the bound $\delta_{\max}$ (Definition 3.5). Each LRE may decrease the spot price by up to $\delta_{\max}$; setting $\delta_{\max} = 0$ eliminates this risk entirely but restricts reconfiguration to unminted supply regions. In all cases, existing holders' floor guarantees and lending positions remain unaffected.

Primary-market-only guarantee. The floor price guarantee applies exclusively to the bonding curve's primary issuance market. Secondary markets (decentralized exchanges, order books) may temporarily trade below P_f. Rational arbitrageurs can buy on the secondary market and redeem on the primary market to capture the spread, which exerts upward pressure on the secondary price toward P_f. However, the speed and completeness of this convergence depend on arbitrageur capital availability and transaction costs. Gas spikes, per-transaction limits, and cross-chain latency can further slow convergence.

Fee-dependent sustainability. The floor rises only when trading and credit activity generates fee revenue that is subsequently directed to the floor reserve. In a sustained zero-activity regime, P_f remains constant—though it never decreases. This dependency can be mitigated by denominating the bonding curve's reserve in yield-bearing assets (e.g., staked ETH, tokenized treasuries): the yield accrues to the reserve independently of trading volume, providing a baseline source of floor appreciation even during prolonged low-activity periods. Such assets must preserve instant redeemability (e.g., liquid staking tokens rather than natively staked assets with withdrawal delays) to maintain the liquidity guarantee of Proposition 3.1. If eligible reserve assets lose instant redeemability under stress (e.g., LST

depegging or redemption caps), sells may require queuing or throttling until liquidity is restored.

Curve expressiveness. The number of piecewise-linear segments is bounded by a configurable parameter $K_{\max}$. Increasing $K_{\max}$ improves the expressiveness of the price function but increases gas costs linearly with the number of segments. Richer curve families (e.g., polynomial or exponential segments) could improve price continuity at the cost of increased verification complexity.

Token compatibility. The current implementation assumes a standard ERC-20 collateral token without transfer fees or rebasing behavior. This is an implementation constraint, not a fundamental limitation of the mechanism: balance-change detection and adjusted accounting can extend support to non-standard token types without altering the reserve invariant or floor monotonicity properties.

Open-ended debt incentives. The credit facility charges a one-time origination fee ϕ at loan creation with no streaming interest, producing open-ended loans. While this simplifies the mechanism, borrowers face no time penalty for leaving debt open indefinitely. The origination fee can be scaled with per-borrower utilization—for instance, increasing ϕ as a borrower's aggregate debt approaches the LTV limit—to discourage over-concentration of reserve withdrawals without compromising the non-liquidatable guarantee. For instance, defining per-borrower utilization as $u_i = d_i / \text{maxBorrow}(\ell_i)$ and setting $\phi(u_i) = \phi_{\min} + (\phi_{\max} - \phi_{\min}) \cdot u_i^2$ creates a convex penalty that remains negligible at low utilization but rises sharply near the LTV ceiling. A related concern is permanently abandoned loans (e.g., lost private keys): locked tokens remain in S_0 indefinitely, inflating the floor supply and thus the collateral cost of future floor elevations via Algorithm 1. A governance-gated buyout mechanism—allowing anyone to repay a dormant loan's debt and burn the freed tokens after a sufficiently long inactivity period—could clear this deadweight without reintroducing oracle risk, since repayment is denominated in the same collateral and the floor price is unaffected.

MEV at step boundaries. Discrete steps create deterministic price transitions at known supply thresholds, which can be front-run by MEV searchers. Buy transactions that cross a step boundary are predictably more expensive per token on the higher step, creating sandwich opportunities analogous to those in constant-product AMMs but concentrated at discrete cliffs. A sufficient condition to make round-trip sandwich attacks strictly unprofitable is to ensure that the fee friction exceeds the step delta: for a step at price p_j with increment Δp and one-way fee rate f (so the round-trip cost is $2f$), the protocol should enforce

$$\Delta p \;<\; 2\, f\, p_j. \tag{20}$$

Under this constraint, the profit from front-running a step crossing is strictly dominated by the fees paid on the buy and sell legs. The protocol's existing fee infrastructure (origination and trading fees) provides a natural enforcement hook; curve designers should parameterize Δp and f jointly to satisfy Equation (20) at every step.

Capital efficiency trade-off. Anchoring LTV to P_f rather than the spot price yields a conservative effective LTV at origination: when the spot price significantly exceeds the floor, the borrowable amount per token is lower than under oracle-based lending. This is the cost of eliminating liquidation risk. Over time, however, as P_f rises through fee accumulation and step absorption, the gap narrows and capital efficiency improves passively without any borrower action—a property absent from oracle-based systems where capital efficiency degrades with volatility.

Idle premium harvesting. When tokens are locked as loan collateral, the premium capital backing those tokens above the floor becomes idle: it cannot be redeemed (the tokens are encumbered) yet it contributes to the reserve. The protocol can harvest this idle premium through an invariant-preserving reconfiguration (Definition 3.5): the premium steps backing locked supply are collapsed to the floor level, freeing surplus virtual collateral that is then injected into the floor segment via Algorithm 1 (step absorption), raising P_f without any external capital injection. The resulting feedback loop is reflexive: a floor rise increases every borrower's capacity via Equation (10), enabling additional borrowing that generates origination fees, which fund further floor elevation. The DABC architecture enables this optimization because it decouples virtual pricing (the curve geometry) from physical reserve management: the reconfiguration is a purely virtual accounting operation that preserves all invariants (Proposition 3.4) while unlocking capital that would otherwise sit idle for the duration of the loan.

Leveraged positions as a design primitive. The recursive leverage loop (Section 4.5) enables a pattern not possible under oracle-based lending: leveraged positions where every loan in the loop is provably safe, available at any point in the curve's lifecycle. Token launches are the most compelling application due to maximal looping efficiency, opening design questions around optimal LTV schedules during presale phases and the interaction between decaying fee multipliers and leverage convergence rates. The premium gap between spot and floor creates a self-reinforcing engine: looping efficiency attracts participants, generating trading and origination fees that fund floor elevation, which in turn narrows the gap and improves capital efficiency for future participants.

Future directions. Several avenues extend the present work. First, formal verification of the core curve mathematics and reserve accounting (e.g., via Certora or Halmos) would strengthen the guarantees beyond fuzz testing. Second, cross-chain deployment raises questions about reserve consistency when collateral resides on multiple networks. Third, automated optimization of the floor-raise cadence—for instance, triggering raises when accumulated fee revenue exceeds a configurable threshold—would further reduce operational overhead. Fourth, deeper integration with secondary markets through protocol-owned liquidity positions could tighten the convergence between secondary prices and the floor. Fifth, the recursive leverage loop introduces *reflexivity*: leveraged buying elevates the spot price and generates fees that raise the floor, which in turn improves capital efficiency for subsequent loops. Section 5.5 presents a first instantiation of dynamic fee multipliers based on Michaelis–Menten kinetics and quadratic size scaling; characterizing the equilibrium dy-

namics of the reflexivity loop under such adaptive fees—including conditions under which it amplifies or dampens—remains open.

7 Conclusion

We have presented debt-aware discrete bonding curves, a mechanism that augments piecewise-linear bonding curves with a monotonically non-decreasing floor segment and a reserve invariant that accounts for outstanding debt. The key insight is that anchoring loan-to-value ratios to an endogenous, monotonically non-decreasing floor price—rather than to an external oracle—eliminates the need for liquidation. Borrowing capacity is derived from a quantity that, by construction, never declines, ensuring that no loan becomes under-collateralized through market movements alone.

We proved that the floor price is non-decreasing under all protocol operations (Theorem 4.1) and that every loan remains solvent without liquidation (Theorem 4.2). Invariant-preserving curve reconfiguration (Definition 3.5) provides a new primitive for adaptive market making on bonding curves, enabling dynamic reshaping of price discovery without violating safety properties.

Comparative analysis against existing floor-price designs and oracle-based lending protocols shows that the mechanism is the first in our survey to combine floor enforcement with integrated lending while eliminating oracle dependency, liquidation cascades, and health-factor monitoring. Capital efficiency improves passively as the floor rises, and redemption-driven supply reduction accelerates future floor appreciation—an antifragile property absent from existing designs.

More broadly, the work demonstrates that DeFi lending need not rely on liquidation cascades as the fundamental solvency mechanism. By coupling token issuance with a deterministic, monotonic price floor, we open a design space in which borrowing is structurally safe and collateral quality improves with protocol usage rather than degrading with market volatility.

References

Aave. Aave protocol whitepaper, version 1.0. Whitepaper, 2020. URL https://github.com/aave/aave-protocol/blob/master/docs/Aave_Protocol_Whitepaper_v1_0.pdf.

Hayden Adams, Noah Zinsmeister, and Dan Robinson. Uniswap v2 core. Whitepaper, 2020. URL https://docs.uniswap.org/whitepaper.pdf.

Hayden Adams, Noah Zinsmeister, Moody Salem, River Keefer, and Dan Robinson. Uniswap v3 core. Whitepaper, 2021. URL https://app.uniswap.org/whitepaper-v3.pdf.

Ajna Labs. Ajna protocol: Peer-to-pool lending without oracles. Whitepaper, 2023. URL https://www.ajna.finance/pdf/Ajna_Protocol_Whitepaper_01-11-2024.pdf.

Baseline Markets. Baseline protocol: Automated tokenomics engine. Protocol Documentation, 2024. URL https://docs.baseline.markets/.

Álvaro Cartea, Fayçal Drissi, Leandro Sánchez-Betancourt, David Siska, and Lukasz Szpruch. Strategic bonding curves in automated market makers. *SSRN Electronic Journal*, 2024. doi: 10.2139/ssrn.5018420.

Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *IEEE Symposium on Security and Privacy (SP)*, 2020. doi: 10.1109/SP40000.2020.00040.

Omer Demirel. Discrete bonding curves. Medium, 2023a. URL https://medium.com/@demirelo/piecewise-constant-bonding-curves-fcc826449acd.

Omer Demirel. Dynamic discrete bonding curves. Medium, 2023b. URL https://medium.com/@demirelo/dynamic-kpi-bonding-curves-55b3bf5602bc.

Omer Demirel. Bonding curves for L1 token economies. Medium, 2024a. URL https://medium.com/@demirelo/bonding-curves-for-l1-token-economies-7afb60f392e1.

Omer Demirel. Protocol-owned automated market makers. Medium, 2024b. URL https://medium.com/@demirelo/protocol-owned-automated-market-makers-0cfdb110f5a3.

Michael Egorov. crvUSD: Llamma and stablecoin design. Whitepaper, 2023. URL https://github.com/curvefi/curve-stablecoin/blob/master/doc/curve-stablecoin.pdf.

Lioba Heimbach and Wenqian Huang. DeFi leverage. Working Paper 1171, Bank for International Settlements, 2024. URL https://www.bis.org/publ/work1171.pdf.

Eyal Hertzog, Guy Benartzi, and Galia Benartzi. Bancor protocol: Continuous liquidity for cryptographic tokens through their smart contracts. Whitepaper, 2017. URL https://resources.bancor.network/pages/BancorProtocolWhitepaper.pdf.

Dennis Kirste, Alexander Poddey, and Ali Sunyaev. Undergirding bonding curves: Supply-sovereign automated market makers enabling liquidity and sustainable financing. *Distributed Ledger Technologies: Research and Practice*, 2025. doi: 10.1145/3716177.

Robert Leshner and Geoffrey Hayes. Compound: The money market protocol. Whitepaper, 2019. URL https://github.com/compound-finance/compound-money-market/blob/master/docs/CompoundWhitepaper.pdf.

MakerDAO. The Maker protocol: MakerDAO's multi-collateral Dai (MCD) system. Whitepaper, 2017. URL https://makerdao.com/whitepaper/DaiDec17WP.pdf.

Nirvana Finance. Nirvana protocol documentation. Protocol Documentation, 2022. URL https://docs.nirvana.finance/.

Olympus DAO. Olympus protocol: OHM is smart money. Protocol Documentation, 2021. URL https://docs.olympusdao.finance/. Includes cadCAD modeling by BlockScience.

Daniel Perez, Sam M. Werner, Jiahua Xu, and Benjamin Livshits. Liquidations: DeFi on a knife-edge. In *Financial Cryptography and Data Security (FC)*, 2021. doi: 10.1007/978-3-662-64331-0_24.

Tim Roughgarden. Transaction fee mechanism design. In *ACM Conference on Economics and Computation (EC)*, 2021. doi: 10.1145/3465456.3467591.

Phoebe Tian and Yu Zhu. Liquidation mechanisms and price impacts in DeFi. Staff Working Paper 2025-12, Bank of Canada, 2025. URL https://www.bankofcanada.ca/2025/03/staff-working-paper-2025-12/.

Timeswap Labs. Timeswap: Permissionless, oracle-less, non-liquidatable fixed maturity lending & borrowing. Whitepaper, 2022. URL https://timeswap.io/whitepaper.pdf.

Algorithm 1 StepAbsorption: Raise the floor price by injecting collateral

Require: Current segments $\Gamma = (\sigma_0, \sigma_1, \ldots, \sigma_{k-1})$; collateral budget $c > 0$; actual issuance supply $S_{\text{actual}} > 0$

Ensure: New segments Γ'; updated floor price $P_f' \geq P_f$

1: $(P_f, S_0) \leftarrow (\text{initialPrice}(\sigma_0),\ \text{supplyPerStep}(\sigma_0))$
2: $S_0 \leftarrow \min(S_0,\ S_{\text{actual}})$ ▷ Cap floor supply at actual issuance
3: $\mathit{capped} \leftarrow (S_0 = S_{\text{actual}})$ ▷ One-way flag: once set, S_0 is frozen
4: $j \leftarrow 1$ ▷ Index into premium segments
5: Load $(p_j, \Delta p_j, q_j, n_j) \leftarrow \sigma_j$
6: **while** $c > 0$ **do**
7: $S_{\text{eff}} \leftarrow S_0$ ▷ Effective supply (already capped by lines 2–3 or 17–18)
8: $\mathit{raisable} \leftarrow c / S_{\text{eff}}$ ▷ Max price increase affordable
9: $\mathit{gap} \leftarrow p_j - P_f$ ▷ Price distance to next premium step
10: **if** $\mathit{raisable} < \mathit{gap}$ **then**
11: $P_f \leftarrow P_f + \mathit{raisable}$
12: **break** ▷ Budget exhausted within gap
13: **end if**
14: $\mathit{cost} \leftarrow \mathit{gap} \cdot S_{\text{eff}}$ ▷ Collateral to absorb this step
15: $c \leftarrow c - \mathit{cost}$; $\quad P_f \leftarrow p_j$
16: **if** $\neg\, \mathit{capped}$ **then**
17: $S_0 \leftarrow S_0 + q_j$
18: **if** $S_0 \geq S_{\text{actual}}$ **then**
19: $S_0 \leftarrow S_{\text{actual}}$; $\quad \mathit{capped} \leftarrow \mathit{true}$
20: **end if**
21: **end if**
22: $n_j \leftarrow n_j - 1$
23: **if** $n_j = 0$ **then**
24: $j \leftarrow j + 1$
25: **if** $j \geq k$ **then break** ▷ No more premium segments
26: **end if**
27: Load $(p_j, \Delta p_j, q_j, n_j) \leftarrow \sigma_j$ ▷ Advance to next segment
28: **else**
29: $p_j \leftarrow p_j + \Delta p_j$ ▷ Advance within current segment
30: **end if**
31: **end while**
32: $c_{\text{actual}} \leftarrow R(\Gamma', S) - V$ ▷ Exact collateral via reserve function
33: Construct Γ': new floor $\sigma_0' = (P_f, 0, S_0, 1)$, followed by the remaining (possibly partially consumed) premium segments
34: **return** $(\Gamma',\ c_{\text{actual}})$